



Full length article

Memory-driven deep-reinforcement learning for autonomous robot navigation in partially observable environments

Estrella Montero ^{a, ID}, Nabih Pico ^{b, d, ID}, Mitra Ghergherehchi ^{a, ID, *}, Ho Seung Song ^{c, ID, *}^a Sungkyunkwan University, Department of Electrical and Computer Engineering, Natural Sciences Campus, Suwon 16419, South Korea^b Sungkyunkwan University, Department of Mechanical Engineering, Natural Sciences Campus, Suwon, South Korea^c Catholic Kwandong University, Department of Electronic Engineering, Gangwon-do, South Korea^d Escuela Superior Politecnica del Litoral, ESPOL, Facultad de Ingeniería en Electricidad y Computación, Guayaquil 09-01-5863, Ecuador

ARTICLE INFO

Keywords:

Autonomous robots
Reinforcement learning
Collision avoidance
Human–robot interaction

ABSTRACT

Service robots with autonomous navigational capabilities play a critical role in dynamic contexts where safe and collision-free human interactions are important. However, the unpredictable nature of human behavior, the prevalence of occlusions and the lack of complete environmental perception due to sensor limitations can severely restrict effective robot navigation. We propose a memory-driven algorithm that employs deep reinforcement learning to enable collision-free proactive navigation in partially observable environments. The proposed method takes the relative states of humans within a limited FoV and sensor range as input into the neural network. The model employs a bidirectional gated recurrent unit as a temporal function to strategically incorporate the previous context of input sequences and facilitate the assimilation of the observations. This approach allows the model to assign greater attention to intricate human–robot relations, allowing a better understanding of the ever-changing dynamics within an environment. Simulations and experimental outcomes validate the efficacy of the policy-based navigation approach. It achieves superior collision avoidance performance compared to representative existing methods and exhibits efficient navigation by incorporating the limitations of sensors during training.

1. Introduction

Progress toward autonomous driving systems is currently the subject of intensive research programs. These systems are designed to navigate urban environments while prioritizing safety, efficiency, and intelligent decision-making. This demand has arisen due to workforce scarcity and a growing preference for non-contact services [1]. Robots to move autonomously and efficiently require tackling several challenges, such as building a map of their surroundings, social norm compliance, and real-time decision-making [2]. Perceiving complex environments, particularly unfamiliar and moving pedestrians, acquiring useful information, and generating a policy that enables autonomous navigation are further challenges for these mobile robots.

As a newly developed technology, deep reinforcement learning (DRL) algorithms have conceptualized robot navigation within crowded environments as a Markov decision process (MDP) [3–6]. Through deep neural networks, RL-based methods capture implicit reciprocal movement characteristics of humans and process all their pertinent states, including positions and velocities [5,7]. The acquired knowledge of the environment is then integrated, leading to developing a behavior

policy for the robot. Nevertheless, most DRL algorithms assume precise and complete knowledge of human states and the environment, which can be unrealistic in real-world applications. Because human behavior is unpredictable due to the effect of physical conditions, ongoing activities, and individual size on movement speed, modeling these states precisely is challenging [4]. Some crowd navigation techniques overlook spatial occlusions caused by obstruction of individuals and restricted visibility caused by sensor characteristics [8]. Inherent limitations in sensor capabilities, spatial occlusions, and the dynamic nature of the surrounding area render complete observation by mobile robots a challenge [8,9]. Previous works have explored occlusion-aware planning algorithms [8,10], have addressed navigating robots in environments with limited visibility [9] and explored the effect of a limited FoV [11]. However, many existing approaches are limited to structured environments, such as intersections or settings with few pedestrians, or have not been rigorously tested in real-world conditions. In contrast, crowd navigation involves highly dynamic and unstructured scenarios where human agents exhibit unrestricted mobility, and the reasoning of occlusion is more challenging. This increases the

* Corresponding authors.

E-mail addresses: mitragh@skku.edu (M. Ghergherehchi), hssong@cku.ac.kr (H.S. Song).<https://doi.org/10.1016/j.jestch.2024.101942>

Received 3 July 2024; Received in revised form 28 December 2024; Accepted 30 December 2024

Available online 24 January 2025

2215-0986/© 2025 The Authors. Published by Elsevier B.V. on behalf of Karabuk University. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

level of uncertainty in the modeled environment, and robots are often unable to perform intelligent inferences within the robot's inherent FoV constraints, particularly when these algorithms are deployed in the real world. Bidirectional gated recurrent unit (BiGRU) modules have recently gained significant popularity in deep learning tasks. Initially employed in natural language processing, BiGRU models have been utilized in navigation fields for movement prediction and trajectory estimation [12–14].

We propose integrating BiGRU into a DRL framework specifically designed for navigating a pedestrian-rich, partially observable environment with a limited field of view (FoV) and sensor range. Our method enhances the BiGRU's sequential processing with an attention mechanism. This allows the robot to focus on the most critical human–robot interactions in the environment and guarantees optimal robot performance as the number of humans in the environment increases. The combination of BiGRU and attention techniques, tailored to sensor limitations, optimizes decision-making in partially observable scenes.

Most existing DRL-based, socially aware navigation methods assume the robot has complete information about all the human states in the surroundings during the entire navigation process. However, this is impractical for real-time applications because a robot can only estimate information on position and speed within a sensor range.

This research's significant contribution is outlined below:

1. A novel DRL framework enables a robot to take action in environments with a limited FoV and short sensor range. To effectively handle real-world scenarios, the robot is trained with both narrow FoVs of 120° and full panoramic FoV of 360° while operating within a sensor range of 3 m.
2. The proposed tool discerns among the current states of humans by leveraging the robot's location and the human's estimated location while factoring in human velocity. Our approach introduces a memory-based DRL, leveraging BiGRU to retain observations from sequential input, and an action-based DRL based on attention mechanisms to understand the environment's ever-changing dynamics better. Integrating BiGRUs with attention mechanisms specifically adapted for sensor constraints enhances decision-making in partially observable environments.
3. In a simulated setting, the proposed method is contrasted with advanced navigational methods. The outcomes of the simulation show that the proposed approach yields superior success and collision rates. A physical test employing a Turtlebot robot confirms that the suggested policy can be implemented successfully in real-world situations.

2. Related work

Since the area of robot navigation in crowded environments has received attention, classical techniques used geometric concepts to modify the velocity vector based on each agent's current position [15]. Social Force [16] presented a navigational method that partially solves the “shortsightedness” issue with robots by using a social force model to forecast pedestrian movements. However, this reaction-based approach does not consider the potential implications of the current motion and how future states may affect the current decision, facing “freezing robot” problems in complex scenarios [17].

2.1. DRL for crowd navigation

Recently, robot navigation based on DRL has received increasing research attention. These approaches involve deploying a deep neural network to extract environmental properties and take an optimal course of action, determined through interactions with the environment and guided by the accumulation of long-term rewards [5,18–20]. Using sensor-level collision avoidance, Long et al. [18] introduced an end-to-end learning framework that directly maps raw sensor data to robot

control inputs. Yao et al. [21] provided a sensor map and incorporated pedestrian maps to enable more nuanced collision avoidance strategies. Following this line, Huang et al. [22] proposed a model that fuses raw LiDAR and camera data to enhance perception capabilities. These algorithms learn directly from raw sensor data to make decisions without relying on an intermediate process for detecting dynamic objects, their positions, or velocities. To alleviate the use of only raw sensor data, other learning-based methods demonstrate a greater ability to mimic human-like behavior by acquiring implicit traits that encode social interactions. Based on this foundation, Chen et al. [7] introduced collision avoidance via DRL, which enables socially aware navigation by converting online calculations into offline value-network calculations. When the number of pedestrians varies, this method exhibits constraints in scalability due to the inherent rigidity of the input format. Everett [23] pioneered the application of the long short-term memory reinforcement learning (LSTM-RL) technique, which aggregates state information and interactions between agents by encoding pairs representing their respective states. A self-attention reinforcement learning (SARL) module was proposed by Chen et al. [5] to help learn about the impact of nearby pedestrians. Despite the improved safety offered by these algorithms, concerns about partial environmental observations mean the resulting strategies could be conservative. All these techniques use neural networks to extract features relevant to human–robot spatial interaction. However, disregard for pedestrians' unexpected velocities due to their varied physical conditions and an inability to acquire adequate pedestrian information due to occlusions and the nature of the environment make it difficult to ensure human safety and collision-free navigation. For safety precautions and efficient robot navigation, dynamic warning zones [4] can be created around people according to their size, velocity, and stride length to guarantee collision-free navigation. Previous studies often assume the robot has access to complete ground-truth perception during training; however, in real-world scenarios, FoV and sensing range are critical constraints that limit the accurate perception of the environment and surrounding objects. Some works have explored challenges associated with limited FoV sensing in various applications, including formation control and multi-target tracking [24]. The tracking problem under limited FoV sensing is addressed in [25,26], which considers decentralized multi-robot systems with limited FoV perception. Some approaches used Partially Observable Markov Decision Processes (POMDPs) to explicitly handle hidden states during planning under occlusions. However, they often require prior knowledge of the number of occluded agents, limiting their scalability to large-scale scenarios [27]. Other efforts have addressed navigating robots in crowded environments with limited visibility [9] and explored the effect of a limited FoV [11]. Following a similar approach, [8] proposes strategies for partially or occluded environments. By employing map-based DRL, some algorithms handle static and dynamic occluded environments [8,11]. In recent approaches, the use of BiGRU emerged as a prominent technique in deep learning, particularly for tasks involving sequential data. Initially applied extensively in natural language processing, BiGRUs are adopted in navigation applications such as radar tracking and multi-target prediction [12–14].

This study extends the memory-based DRL application by integrating BiGRU into the domain of robot navigation in environments with limited FoV. The BiGRUs capture temporal dependencies of the environment, retaining past context for better decision-making. Our method incorporates action-based DRL based on attention mechanisms to emphasize human–robot interactions within the robot's limited sensor range and FoV of 120° and 360°. Integrating BiGRUs with attention mechanisms adapted for sensor constraints enhances decision-making in partially observable environments.

3. Methodology

This study formulates the robot's navigational statement as an MDP within the RL method. Subsequently, this section presents the memory-driven network tailored to enable robotic navigation in environments with limited observability.

3.1. Statement of the problem

The robot's task is to approach a goal location while navigating in a partially observable environment characterized by a restricted FoV and short sensor range. Despite having restricted visibility, the robot must navigate safely to its objective, avoiding dynamic agents without any collision.

Within a partially observable environment, the interaction between agents is characterized by the current state vector, $^t\mathbf{s}$, and the feasible action set, $\mathbf{a} \in A$. The robot agent then perceives the environment at a discrete time step, t , and selects $^t\mathbf{a}$, based on the policy function, π . At each state, the objective is to determine the best course of action, i.e., $\pi^*(^t\mathbf{s})$, maximizing the robot's expected future reward. In order to accomplish the agent's objective, the optimal value function V^{π^*} is then calculated as follows:

$$V^{\pi^*}(^t\mathbf{s}) = \mathbb{E} \left[\sum_{i=0}^T \gamma^i v_{\max} R(^{t+i}\mathbf{s}, \pi^*(^{t+i}\mathbf{s})) | ^t\mathbf{s} \right], \quad (1)$$

where $R(^{t+i}\mathbf{s}, \pi^*(^{t+i}\mathbf{s}))$ denotes the expected reward after transitioning to a new state $^{t+i}\mathbf{s}$. The discount factor within the range $(0, 1)$ is denoted by γ , and T indicates when an episode concludes due to a collision, timeout, or accomplishing the goal. Instead of using a manually defined discount factor, the maximum velocity $v_{\max}$, is a weighted parameter that regulates γ . Therefore, $\pi^*(^t\mathbf{s})$ is trainable and estimated from V^{π^*} :

$$\begin{aligned} \pi^*(^t\mathbf{s}) = \operatorname{argmax}_{\mathbf{a} \in A} R(^t\mathbf{s}, \mathbf{a}) \\ + \gamma^{\delta t v_{\max}} \int_{^t\mathbf{s}} P(^t\mathbf{s} | ^t\mathbf{s}, ^t\mathbf{a}) V^{\pi^*}(^t\mathbf{s}) d^t\mathbf{s}, \end{aligned} \quad (2)$$

where P represents the transition model, which is the probability of reaching a new state at $t' = t + \delta t$, based on the action chosen by the robot from, A .

The interactive state space for the robot and human includes the current velocity $\mathbf{v} = [v_x, v_y]$; the maximum velocity $v_{\max}$; the location of each agent coordinates $\mathbf{p} = [p_x, p_y]$; the radius, r ; and the turn angle, ψ .

$$^r\mathbf{s} = [\mathbf{p}_r, \mathbf{v}_r, r_r, \mathbf{g}, v_{\max}, \psi_r], \quad (3)$$

$$^h\mathbf{s} = [\mathbf{p}_m, \mathbf{v}_m, r_m]. \quad (4)$$

At every t during the interaction, the joint state, $^j\mathbf{s}$, represents the system's interactive state. This $^j\mathbf{s}$ combines the separate states of the robot and the human within the system. This study further defines a robot-centric frame to reduce the uncertainty in parameterizing $^t\mathbf{s}$. This involves transforming all agents' positions and speeds into coordinates centered around the robot, where the robot's location serves as the origin and the x -axis points toward its intended destination. Thereby, the spatio-temporal states of both the robot and human can be defined as follows:

$$^r\mathbf{s} = [\mathbf{v}_r, d_{rg}, r_r, v_{\max}, \psi_r], \quad (5)$$

$$^h\mathbf{s} = [\mathbf{v}_m, \mathbf{p}_m, r_m, d_m, r_m + r_r, \psi_m], \quad (6)$$

where $d_{rg} = \|\mathbf{p}_r - \mathbf{g}\|_2$ points from the robot's current location, $\mathbf{p}_r$, toward its destination, $\mathbf{g}$, which is defined as $\mathbf{g} = [g_x, g_y]$. $d_{ms} = \|\mathbf{p}_r - \mathbf{p}_m\|_2$ denotes the Euclidean distance between the robot and its m th neighbor's location. The joint state is then given as:

$$^j\mathbf{s} = [^r\mathbf{s}, ^1\mathbf{s}, \dots, ^h\mathbf{s}, \dots, ^m\mathbf{s}]. \quad (7)$$

The state space captures the relevant features of the environment and is represented as a matrix of dimensions $m \times 14$, where m denotes the number of human agents within the robot's observable state. The matrix includes seven features from the robot's state (position, velocity, goal, robot's radius, preferential velocity, and orientation), as described in (3). It also includes five features from the state of each human agent (position, velocity, radius), as shown in (4). After transforming the

state into a robot-centric frame, the matrix dimensions become $m \times 13$. The first six features are denoted by (5). The remaining seven features correspond to the human state, as is depicted in (6). This structured matrix representation is the internal encoding of the state space. It serves as input to the DRL framework, allowing the robot to predict actions that maximize its long-term rewards.

The memory pool is a buffer and is organized as a list of tuples [state, value]. The state as mentioned earlier, refers to the structured matrix representation of the environment, while the value represents a numerical estimate of the expected cumulative reward associated with that state, as defined by (1). These stored state-value pairs are used to train the neural network, enabling it to approximate a general value function capable of evaluating any possible state.

3.2. Action space

The action space defines the set of possible actions the robot can execute at each timestep. The robot's model is unicycle, and its action, $\mathbf{a}$, is the velocity, $\mathbf{v}_r = [v_r, w_r]$. The linear velocity, v_r , is distributed using an exponential distribution across $N = 5$ speed samples. The angular velocity, w_r , is sampled across $N = 7$, with a maximum angular velocity of $\pi/4$. The action space is represented by the set of all feasible combinations of $[v_r, w_r]$ pair:

$$v_r = \left[\frac{e^{\left(\frac{i}{N}\right)} - 1}{e - 1} v_{\max} \right]; i = 0, 1, \dots, N \quad (8)$$

$$w_r = \left[\pm \frac{\pi}{4} \cdot \frac{e^{\left(\frac{i}{N}\right)} - 1}{e - 1} \right]; i = 0, 1, \dots, N \quad (9)$$

3.3. Reward function

In RL, the reward formulation serves as a scheme of incentives that guides the agent's decision-making process. It involves rewarding the robot for desirable actions toward achieving its objective and penalizing it for undesirable actions. The agent aims to maximize the expected cumulative reward $R(^t\mathbf{s}, \mathbf{a})$ over time, as outlined in (2). Similar to [4,5,28–31], we formulate the reward function to provide a significant incentive for goal achievement, i.e., $(p_x - g_x, p_y - g_y) < r_r$, and penalize it for collision, $d_m < 0$:

$$R_{nav} = \begin{cases} 10, & \text{if } (p_x - g_x, p_y - g_y) < r_r, \\ -0.25, & \text{if } d_m < 0, \end{cases} \quad (10)$$

To ensure safe navigation and maintain a comfortable distance during navigation, similar to [4], a proximity penalty, R_{prox} , is incorporated to discourage the robot from getting too close to agents, i.e.,

$$R_{prox} = -0.1 + \frac{d_{ms}}{2}, \text{ if } 0 < d_{ms} < k_p, \quad (11)$$

where d_{ms} denotes the smallest observed safe distance between the agents, and k_p is a discomfort factor set to 0.2.

When t exceeds the maximum time set for the episode and the robot has not reached its desired final position, a reward for timing out, R_{time} , is enforced, and a penalty of -10 is received [29]. Similar to [30,31], an additional reward is implemented to evaluate continuous progress toward the goal, R_{prog} . The robot is rewarded for getting close to it and penalized for moving in other directions and is given as:

$$R_{prog} = 1 - \frac{d_{rg}[t-1]}{d_{rg}[t]}, \quad (12)$$

where $d_{rg}[t-1]$ and $d_{rg}[t]$ are the magnitudes of corresponding distances separating the robot from its goal between consecutive time steps. The R_{prog} encourages the robot to continue moving toward the objective while shortening the arrival time. The reward function is a combination of the individual rewards described previously. It is defined as

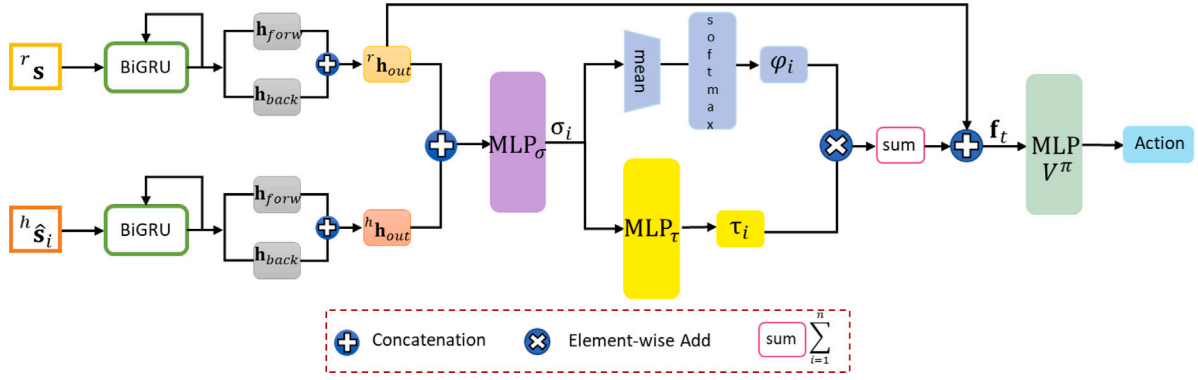


Fig. 1. Design of the memory-driven network framework. The observed human and robot states are input to BiGRU layers to process sequential input and retain temporal dependencies. Attention mechanisms prioritize critical interactions among detected agents. The value function is then computed using an MLP layer for decision-making.

follows:

$$R^t(\mathbf{s}, \mathbf{a}) = \begin{cases} 10, & \text{if reached the goal} \\ -0.25, & \text{if collision} \\ -10, & \text{if time out} \\ -0.1 + \frac{d_{ms}}{2}, & \text{if proximity} \\ 1 - \frac{d_{rg}[t-1]}{d_{rg}[t]}, & \text{otherwise,} \end{cases} \quad (13)$$

3.4. Policy neural network architecture

The DRL framework depicted in Fig. 1 empowers the robot to execute action plans within partially observable environments. This innovative, memory-driven approach combines BiGRUs, multi-layer perceptions (MLPs), and an attention mechanism to handle high-dimensional input and extract meaningful patterns within the robot's FoV. To sustain the acquired awareness over time, the proposed model employs bidirectional GRU. The BiGRU consists of two GRUs that process the sequences in forward and backward directions. The GRU uses internal gating mechanisms (update and reset gates) to manage the flow of information. The update gate regulates how much of the previous data is carried forward, while the reset gate determines how much of the prior state should be disregarded. By retaining relevant information through the gates, the BiGRU can make better-informed decisions based on previous observations. It is well-suited for environments where obstacles temporarily move out of the robot's FoV. In a bidirectional structure, the forward layer processes the input normally, while the backward layer processes it in reverse order. It enables the model to handle occlusions more effectively by leveraging historical data. The proposed framework consists of three stages: spatial feature extraction, states process, and value prediction.

3.4.1. Spatial feature extraction

The FoV, which can be depicted as a small circular region enclosing the robot, is limited to a FoV between 120° and 360° . To obtain the spatial features of the humans localized within the FoV of the robot, a relation, d_{it} , representing the position, velocity, and orientation of the agents is taken into consideration as a tool to compute the inputs of human states to the neural network. The value of d_{it} is set to less than the limited sensor range of the robot and is calculated by:

$$\begin{aligned} d_{it} &= d_m + d_{est}, & d_{it} &\leq range \\ d_m &= \|\mathbf{p}_r - \mathbf{p}_m\|_2, \\ d_{est} &= \|\mathbf{p}_{m(t-1)} + \mathbf{v}_m \cdot \delta t - \mathbf{p}_r\|_2, & \forall_m &\in [1, i], \end{aligned} \quad (14)$$

where d_{est} denotes the robot's estimated distance to a human by considering the past position at $[t-1]$ and the human's current velocity.

3.4.2. States process

The observed human states defined by $h_{\hat{s}}$ and the robot state are passed through two separate BiGRUs. The BiGRU captures temporal dependencies within the sequences of varying lengths in the robot's sensor range, allowing the model to retain relevant trajectory information over time. After the BiGRU processes each human's state relative to the robot the past movements and future trajectories are incorporated. Conversely, the BiGRU processes the robot's state information to provide a temporal context for its trajectory. The BiGRU layer has two hidden states, h_{forw} and h_{back} , with a dimension of 20, representing the forward and backward passes through the network, respectively. These hidden states are then merged to generate the respective outputs of BiGRU, $h_{h_{out}}$ as the human state and $r_{h_{out}}$ as the robot state:

$$h_{h_{out}} = \text{GRU}([(h_{back}, h_{forw})_{t-1}], h_{\hat{s}_i}), \quad (15)$$

$$r_{h_{out}} = \text{GRU}([(h_{back}, h_{forw})_{t-1}], r_s). \quad (16)$$

This model used the previous context of input sequences to allow for a more complete understanding of the dynamic relationships within the environment. The outputs from the BiGRU layers were then concatenated to form an embedding input, which was passed through a multi-layer perception (MLP_σ) of dimensions (150, 100). The formula for the computation of its block generates an intermediate representation of the environment σ_i :

$$\sigma_i = \text{MLP}_\sigma([h_{h_{out}}, r_{h_{out}}]). \quad (17)$$

For every input sequence of $\sigma_i \in \mathbb{R}^{n \times D_\sigma}$, the mean pooling module can be computed and fed concurrently into another (MLP_τ) of dimensions (100, 45) to calculate the attention scores as follows:

$$\phi_i = \text{softmax}(\sigma_i, [(1/i) \sum_{m=1}^i \sigma_m]), \quad (18)$$

$$\tau_i = \text{MLP}_\tau(\sigma_i), \quad (19)$$

where ϕ_i is the attention score derived through a ReLU activation and a softmax function, indicating the relevance of each neighboring element to the robot. τ_i is the intermediate feature representation of the human-robot interaction vector.

An element-wise multiplication of the self-attention module calculated in (18) and τ_i is performed to generate a weighted sum, $\mathbf{f}_t$, that represents the final features for the crowd, as delineated in (20). The weighted feature calculation focuses on more relevant agents based on their proximity and movement, enhancing the model's ability to respond dynamically to interactions within its immediate surroundings.

$$\mathbf{f}_t = \sum_{i=1}^n (\phi_i \cdot \tau_i). \quad (20)$$

Algorithm 1: Memory-Driven Training

```

1 Assign neural network parameters ;
2 Adjust hyperparameter  $\alpha$  ;
3 Setting  $N_{epochs}$ ;
4 Set batch size;
5 Initiate experience memory  $E \leftarrow \text{Demo}$ ;
6 Initiate  $V^\pi \leftarrow E$ ;
7 Set  $\hat{V}^\pi \leftarrow V^\pi$ ;
8 for episode=1,2,...,N do
9   Initiate  $s_{t=0}$  randomly;
10  repeat
11     $\mathbf{a} \leftarrow \begin{cases} \text{random action } () \\ \text{argmax}_{\mathbf{a}_i \in A} R(\mathbf{s}_t, \mathbf{a}_i) + \gamma^{dt_{max}} V^\pi(\mathbf{s}_{t+\delta t}) \end{cases}$ ;
12    Store  $E \leftarrow (\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+\delta t})$ ;
13    Mini-batches are sampled from  $E$ ;
14    Optimize  $V^\pi$  by gradient descent
15  until  $\mathbf{p}_r = \mathbf{g}$ , collision or  $t \geq t_{max}$ ;
16  Update  $\hat{V}^\pi \leftarrow V^\pi$ 
17 end
18 return  $V^\pi$ 

```

3.4.3. Value prediction

In the final stage, the weighted feature and the robot state output $r_{h_{out}}$ are concatenated and input into a multi-layer perception (MLP_V) of dimensions (150, 100, 100, 1) to estimate the value function value, V^π , where $\mathbf{W}_V$ denotes the network weights, as shown in (21).

$$V^\pi = \text{MLP}_V((\mathbf{f}_t, r_{h_{out}}); \mathbf{W}_V). \quad (21)$$

The value network's training process involves adjusting the model parameters to learn to assign appropriate weights. The training procedure is outlined in Algorithm 1 in a pseudo-code format. From steps 1 to 7, the model undergoes a series of demonstration episodes and epochs. This process is fine-tuned by adjusting learning rates. In step 11, the epsilon-greedy strategy ϵ balances exploration and exploitation during action selection. With a probability of ϵ , the agent randomly selects actions to explore a wider range of possible actions, promoting diversity in its experiences. In contrast, with a probability of $1 - \epsilon$ the optimal action is derived from experiences stored in the memory pool, leveraging prior knowledge to maximize rewards. In step 13, mini-batches of sampled experiences, $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+\delta t})$ are used to reinforce successful strategies and address mistakes without requiring constant interaction with the environment. The process iterates until the terminal state is achieved (step 15).

The memory-driven network, composed of BiGRU layers, MLPs, and attention mechanism, extracts and retains relevant temporal and spatial features from observed agent interactions. These processed features compute the value function V^π , enabling the robot to select optimal actions to maximize expected rewards. Algorithm 2 describes the overall architecture of the proposed memory-driven DRL framework. From steps 4 to 9, the robot and human states are passed through BiGRU layers to retain temporal context and an attention mechanism to prioritize critical interactions. The features are processed to estimate the value function, which is used for action selection in the DRL framework (steps 11–22). From steps 14 to 19, the network is trained using the DRL framework, incorporating rewards based on the robot's performance. Finally, the trained memory-driven policy is evaluated (steps 21–23).

4. Experiments and results

This section presents the results of an extensive set of simulations designed to benchmark the proposed memory-driven policy against state-of-the-art navigation algorithms. The experiments are conducted

Algorithm 2: Memory-Driven DRL framework

```

1 Input: Robot and Human states  $s_t$  at time  $t$ 
2 Output: Optimal Policy  $\pi^*(\mathbf{s})$ 
3 //Memory-Driven Network
4 for each observation at time  $t$  do
5    $d_{it} = d_m + d_{est}$ ,  $d_{it} \leq \text{FoV range}$ 
6   Process robot and human states using BiGRU Layers;
7   Compute attention scores  $\phi_i$  using the Attention Mechanism;
8   Extract weighted features  $\tau_i$  based on attention scores;
9 end
10 //Value Estimation
11 Combine outputs  $\phi_i, \tau_i$ ;
12 Estimate Value Function  $V^\pi$ ;
13 //DRL framework
14 while episodes < training episodes do
15   Algorithm 1 to perform DRL Training;
16   Select action  $\mathbf{a}$  using  $\epsilon$ -greedy strategy;
17   Execute action and observe reward  $R(\mathbf{s}, \mathbf{a})$ ;
18   Update policy and target network ;
19 end
20 //Action Execution
21 Testing 3,000 episodes
22  $\mathbf{a} \leftarrow \arg \max_{\mathbf{a}_i \in A} (R(\mathbf{s}, \mathbf{a}) + \gamma^{dt_{max}} V^\pi(\mathbf{s}_{t+\delta t}))$ ;
23 Update states and repeat;

```

in diverse simulated environments, with the robot's FoV varying between 120° and 360° and interacting with dynamic and static elements.

4.1. Experimental setup

The simulated environments include 15 humans modeled with circles of radii $r_{ih} \in [0.2, 0.6]$ m. The human movements are derived from (ORCA) system [15] with random velocities of [0.5, 1.8] m/s, according to their constantly changing intents and different traits [4]. The motions of the human agents are influenced by each other, but their motions are unresponsive to the robot. A circle of $r_r = 0.3$ m is visualized as the robot. The FoV is modeled as a small circular region and a cone-shaped for 360° and 120°, respectively. The robot can only sense objects to a limited distance of 3 m within its FoV. The FoV around the robot mimics the limited range of cameras, radars, and ray-based sensors such as LiDAR. The starting position of the agent is determined from a 6 m × 6 m square.

The training and testing procedures were implemented using Pytorch module. Initially, the model finished 3000 in demonstration episodes. Subsequently, the model was trained for 400 epochs using the gradient descent method with a learning rate, α , of 0.01, as outlined in Algorithm 1. A learning rate of 0.001 was employed during the training, and the robot underwent training for 10,000 episodes. The discount factor γ was set to 0.9. The greedy strategy's exploration rate, ϵ , declined linearly from 0.5 – 0.1 over the initial 4000 episodes, after which it stayed constant for the remaining episodes. After the initial 1000 episodes, mini-batches, each consisting of 100 samples, were utilized to augment the value network. For comparison, we selected LSTM-RL [23], SARL [5], and ESA [9] as representative methods due to their varying approaches to crowd navigation and handling partial observability. The LSTM-RL method utilizes the LSTM network to pair the robot with all humans in dynamic crowd scenarios. This method structures input data by arranging agents in order of proximity, with the closest agent having the most immediate influence on the robot's action. The SARL approach enhances social interaction modeling by applying a self-attention technique to capture pairwise interactions

Table 1

Performance comparison in static and dynamic environments with FoV of 120° and 360°.

Metrics	FoV of 120°						FoV of 360°					
	SR	CO	TD	TT	T_s	DR	SR	CO	TD	TT	T_s	DR
LSTM-RL	0.64	0.26	10.49	12.11	0.36	2.56	0.61	0.37	9.17	9.83	0.43	0.56
SARL	0.50	0.35	9.94	11.15	0.41	2.03	0.66	0.34	9.92	11.42	0.39	0.75
ESA	0.68	0.31	11.22	14.53	0.24	0.80	0.74	0.25	11.71	15.16	0.20	0.53
OURS	0.78	0.20	9.81	10.77	0.43	0.37	0.93	0.07	8.46	10.71	0.44	0.25

Table 2

Performance comparison in crowded environments with FoV of 120° and 360°.

Metrics	FoV of 120°						FoV of 360°					
	SR	CO	TD	TT	T_s	DR	SR	CO	TD	TT	T_s	DR
LSTM-RL	0.49	0.50	10.41	10.69	0.44	2.31	0.62	0.35	8.91	10.77	0.43	0.78
SARL	0.63	0.36	11.52	14.75	0.22	2.11	0.67	0.33	9.12	11.71	0.38	0.83
ESA	0.61	0.38	11.28	14.10	0.26	0.67	0.71	0.29	11.68	15.19	0.20	0.59
OURS	0.80	0.21	10.19	10.25	0.46	0.64	0.90	0.10	8.56	10.38	0.45	0.41

between the robot and surrounding humans. The ESA method incorporates both spatial and attention graphs to model the relationship between the robot and the human agents. Similar to the LSTM-RL method, it sorts input data in reverse order of distance to prioritize nearby agents, but it further refines the interaction modeling by capturing more nuanced spatial relations. The network frameworks are kept as outlined in their original works to ensure consistent comparison.

4.2. Simulated experiments

The weights of the policy network saved during the training were used to assess the navigation performance on a set of 3000 test cases. For a comprehensive evaluation comparison, significant metrics were presented. The success rate (SR) indicates the successful cases that did not result in collisions or in which the robot did not get stuck somewhere during navigation. The collision ratio (CO) denotes the rate of collision between the robot and the obstacles. The traveled distance (TD) refers to the distance the robot covers to reach its destination. The travel time (TT) is the average time for successful navigational efforts. To evaluate the efficiency in terms of time steps, we adopt the T_s score function as is proposed in [32]. This metric is calculated by $T_s = 1 - 2(S_s/T_{max})$, where S_s denotes the number of time steps spent on navigation, and T_{max} is the maximum time steps per episode. The discomfort ratio (DR) is the percentage of time the robot traveled too close to a human agent. To ensure a consistent evaluation across settings, each method was trained separately with a FoV of 120° and 360°. The specified FoV was maintained constant across every episode. The sensing range was set to 3 m to test the collision avoidance ability with restricted information.

4.2.1. Quantitative results

The experiments are conducted in a *hybrid* environment, which includes static and dynamic obstacles, and a *crowd-simulated* scenario. Table 1 and Table 2, respectively, present the results.

The results show that when the robot operated in a *hybrid environment*, limiting its FoV to 120°, the methods significantly decreased its performance. The freezing robot problem was prevalent, and the robot was less likely to reach its goal with the lowest SR. These results can be determined by the fact that LSTM-RL and SARL consider that the robot is fully aware of all human policies and their goals during the entire navigational process, i.e., full visibility of the environment is assumed. On the other hand, the ESA method estimates the states of detected obstacles in reverse order. However, considering that the nearest obstacle has the highest influence is not feasible because additional features such as velocity and orientation must be considered. The experimental findings of the simulation as depicted in Table 1 reveal that our method outperforms the other tree learning baselines across all metrics. The proposed method successfully estimated the human's location within

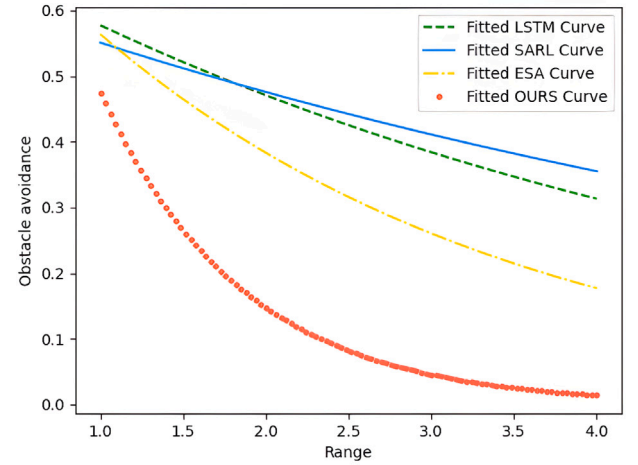


Fig. 2. Comparison of state-of-the-art methods for 360° navigation with different sensing ranges.

the sensing range by capturing dynamically changing features because it uses its acquired awareness as long as necessary, obtaining an SR of 0.93, while remaining safe in terms of CR and DR, at 0.07 and 0.25 respectively.

In the *crowd-simulated* environment, 15 humans were included and the outcomes are presented in Table 2. The effectiveness of existing methods revealed underperformance in environments with a limited FoV and sensing range. These results, as displayed in Table 2, demonstrate the SR is lower with 120° than with 360°, and indicate an increase in the number of collisions. ESA exhibited superior results compared to LSTM-RL and SARL by enhancing the SR to 0.71 and decreasing the CO to 0.29. While the LSTM-RL achieved a shorter navigation time, the registered collision cases demonstrate neglect in avoiding obstacles during navigation, resulting in a higher DR. Although the SARL method analyzed the spatial relationship in the crowd, this method assumed all human-state transitions were well-defined and well-known, resulting in the lowest performance. Compared with the existing approaches, the proposed policy achieved superior performance in SR and CO for all testing scenarios, demonstrating the robot can complete the navigational task with relatively few collisions. In addition, the proposed memory-driven policy exhibited the lowest TT compared to all the methods. This is explained by the fact that our approach was designed to enhance the robot's performance in terms of safe navigation and minimize the arrival time. The lowest DR metric further supports this. As shown in Table 2 and Table 1, our method achieved a higher score, T_s , suggesting it completed navigation more

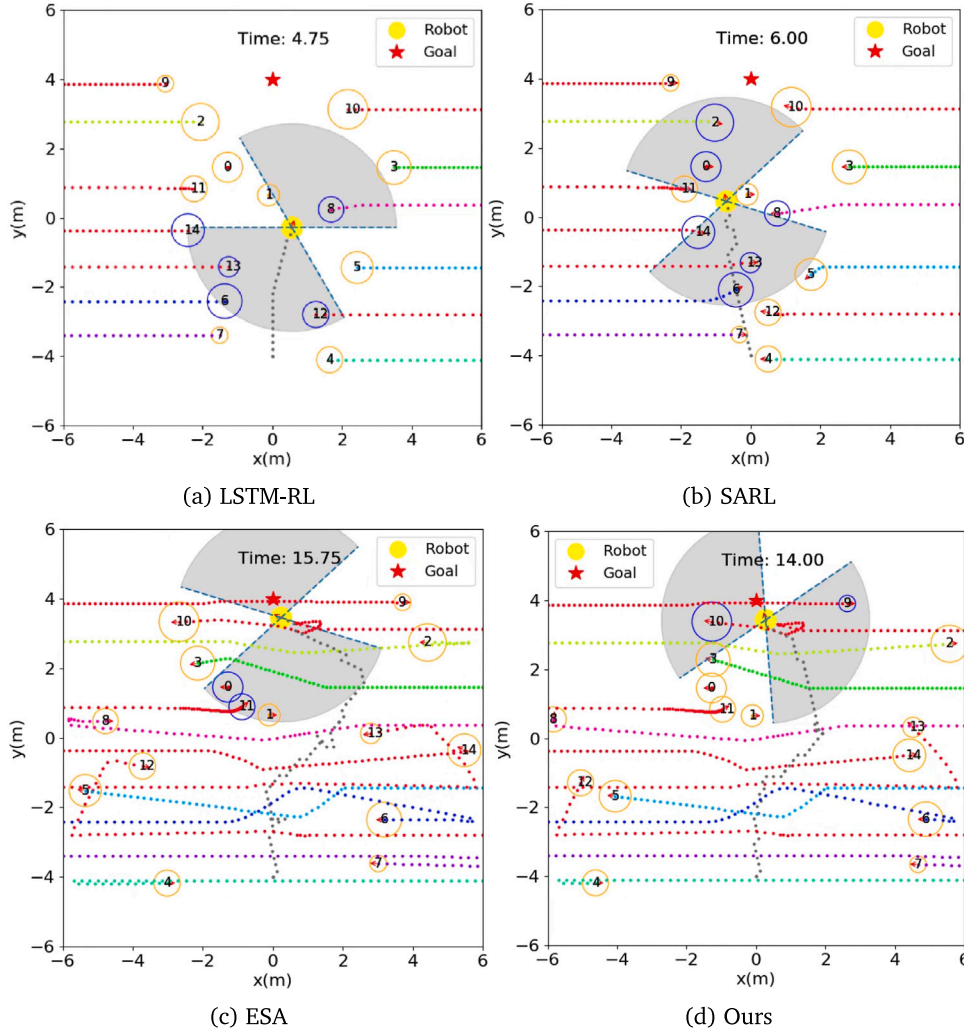


Fig. 3. Trajectory illustrations generated in a test episode with a FoV of 120° on both the front and back sides, involving 15 humans. All simulation tests were conducted under identical conditions, including time steps, starting and goal positions. The humans detected within the sensor range are presented in blue, while unobserved humans are in orange circles.

efficiently.

In addition, we varied the robot's sensing range to 1, 1.5, 2, 2.5, 3, 3.5, and 4 m with a FoV of 360° in the same environment to assess the relationship between collision-free navigation and visibility range. As shown in Fig. 2, with a sensor range of 1 m, these four policies performed similarly, revealing suboptimal outcomes. With a slightly longer range of 0.5 m, the proposed method reduces its risk of collision from 0.5% to 0.2%, while all baseline methods produced higher percentages ($> 0.5\%$). At 2 m and more, the proposed method enabled the robot's decision-making and adapted its obstacle-avoidance capabilities, maintaining percentages closer to 0.05. The LSTM-RL, SARL, and ESA methods do not ensure reliable obstacle avoidance, even at longer sensing.

The proposed policy provides a detailed understanding of the ever-changing dynamics within the environment by identifying the most relevant human states, using, BiGRUs layers to reveal the underlying relationships among the crowd, and predicting future situations. This demonstrates that the proposed policy is robust and enables the robot to make better-informed decisions in partially observable environments.

4.2.2. Qualitative results

In a fair environment in which 15 human courses are the same, the navigation trajectories of various techniques are contrasted in Figs. 3 and 4. The robot's FoV is constrained to 120° and 360° , depicted respectively as a circular sector and a circular region around the robot. The

observed humans within the robot's sensing range are highlighted in blue, while those outside of it are highlighted in orange. As is shown in Fig. 3, the robot was guided to complete a task in a scenario containing 15 humans and static obstacles, the sensor range radius was set to 3 m, and the FoV was 120° . All compared methods successfully navigated to the goal; however, the LSTM and SARL tended to keep moving into crowds directly, resulting in collisions. Operating within a limited FoV, the ESA method waits for humans to move aside, prioritizing collision avoidance. However, this prolongs the time required to accomplish the goal. In comparison, the proposed policy effectively addresses the challenge of a limited FoV and avoids collisions with obstacles. The proposed policy estimates the positions of the obstacles in its current view and captures past observations for a deeper comprehension of the dynamic relationships. This enables the robot to make better predictions and navigate to reach its goal in scenarios with limited visibility.

A 360° FoV robot navigation test case is displayed in Fig. 4. The robot's navigation objective includes reaching its designated goal position while avoiding human concentration. With the LSTM-RL method, the robot took indecisive actions, colliding with the human navigating from behind, as illustrated in Fig. 4(a). The SARL method initially avoided the human clusters but subsequently backtracked, leading the robot to execute additional maneuvers. The robot loses sight of the goal and consequently takes longer to arrive at it, as is shown in Fig. 4(b).

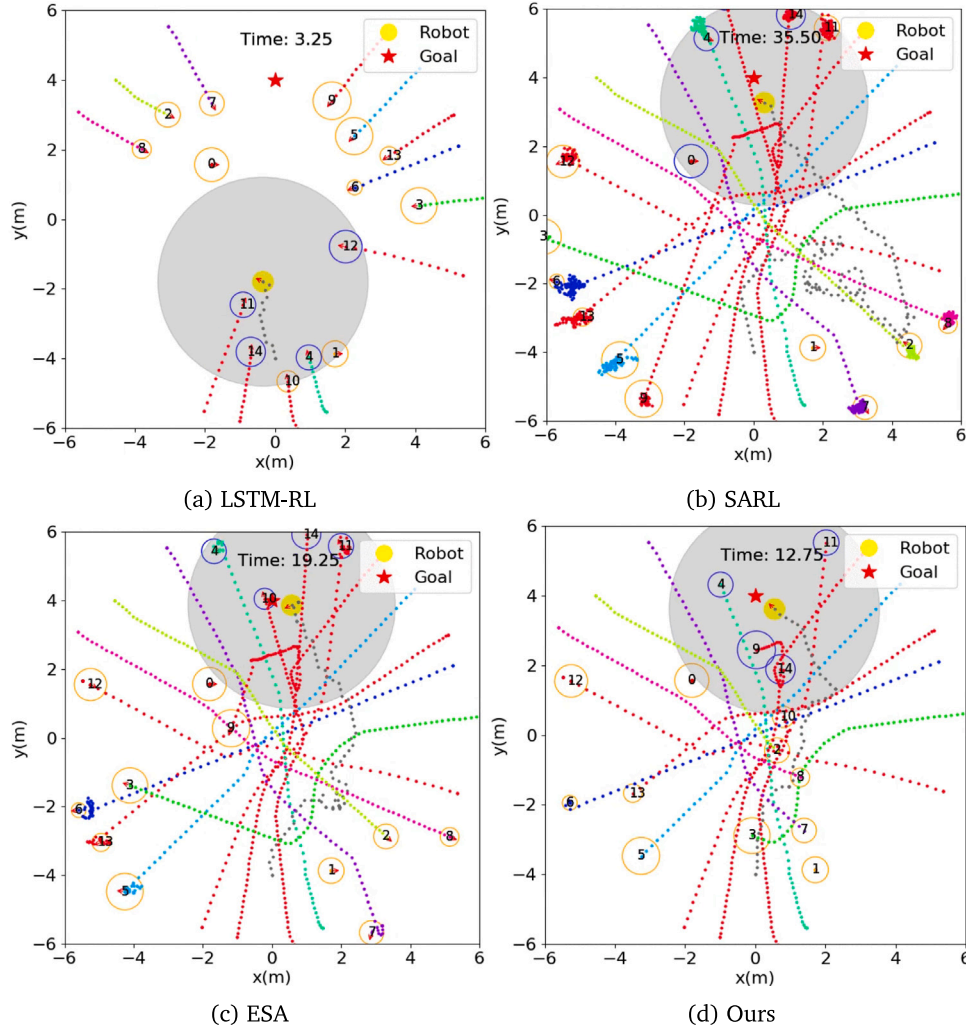


Fig. 4. Trajectory illustrations generated in a test episode with a 360° FoV and 15 human agents. All simulation tests were conducted under identical conditions, including time steps, starting and goal positions. The humans detected within the robot's FoV are in blue, while unobserved humans appear in orange circles.

Fig. 4(c) depicts the robot's trajectory with the ESA method. The robot steered clear of the human clusters but it failed to find an even path. The robot frequently halted and backtracked to avoid the humans, causing the robot to occasionally freeze in place. The ESA model opts for paths that result in prolonged navigation time. In contrast to the scenarios outlined earlier, the proposed policy tactfully veers slightly to the right to avoid areas of high human concentration. The proposed model waits for the humans to move out of its path before moving forward, ensuring collision-free navigation. As illustrated in Fig. 4(d), the method proposed overcame the baseline methods in terms of safety in challenging scenarios, reducing the chance of freezing robot scenarios. Furthermore, this approach reduced arrival time by maintaining an even trajectory. The experiment's outcomes demonstrate that past behavior patterns are key to understanding the potential influence of each person's previous actions on the robot's ability to achieve its goals effectively. Essentially, our memory-based DRL by analyzing how individuals have moved and behaved in the past, can predict how they might act in the future. Our action-based DRL component enhances decision-making, enabling the robot to handle unpredictable changing environments.

5. Gazebo-ROS validation

An evaluation is conducted in the Gazebo-ROS simulator to assess the generalization capability in environments with varying crowd sizes

Table 3

Gazebo-ROS results in an environment with dynamic and static obstacles.

Metrics		SR (%)	CR (%)	TT (s)	TD (m)
First Target = 7 m	SARL	98	60	58.78	13.52
	GDAE	98	70	64.97	14.94
	OURS	99	0	42.23	9.71
Second Target = 6 m	SARL	97	75	76.69	17.64
	GDAE	98	50	79.72	18.33
	OURS	98	10	76.64	17.62
Third Target = 4 m	SARL	97	60	75.87	17.45
	GDAE	55	92	154.75	35.59
	OURS	99	0	35.93	8.26
Fourth Target = 9 m	SARL	45	85	88.68	20.39
	GDAE	85	60	69.23	15.92
	OURS	99	0	52.45	12.06

and to handle high-dimensional inputs from laser scans. For the simulated test in Gazebo, a Turtlebot3 is utilized with a 2D Laser Distance Sensor (LDS) to map the environment and detect obstacles. The maximum velocity of the Turtlebot3 is limited to 0.23 m/s, and its FoV is 360° with a measurement range set to 3.5 m. A Gazebo simulation environment is created to test all methods. This environment is roughly 27 × 14 m and is filled with 22 human and static obstacles (e.g., chairs, tables, and partition walls). A 2D occupancy grid map of the environment is created by employing the *Gmapping* ROS package [33], and for



Fig. 5. Comparative analysis of SARL, GDAE, and our proposed method in a scenario with dynamic and static obstacles in Gazebo-ROS.

the robot's localization, the *amcl* ROS package [34] is utilized. The obstacles are detected using the *Obstacle detector* package [35]. Based on prior training results, the SARL and our proposed method are deployed in the Gazebo-ROS simulator without retraining. An additional method, Goal-Driven Autonomous Exploration (GDAE) [36] is implemented in the Gazebo simulation for comparison. The GDAE method [36] learns generic obstacle avoidance behavior from laser data simulated from ROS, its framework was also kept for a fair comparison. During the evaluation, the robot is tasked with a sequence of four navigation goals within the same scenario. Initially, the robot is assigned a goal located 7 m away. It is given a new target at an additional 6 m upon reaching this point. This process is repeated, with the robot sequentially receiving two distinct goals of roughly 4 m and 9 m. Fig. 5 shows the simulated environment, and the quantitative results are depicted in Table 3.

According to Fig. 5(c), our proposed policy ensures a smoother trajectory of motion during collision avoidance, effectively reaching the target points while minimizing arrival time and reducing the overall distance traveled. Figs. 5(a) and 5(b) reveal that SARL and GDAE methods navigate all sequential target points. However, collisions are highest, generating longer and more tortuous paths. SARL further takes longer to reach the goal and finally freezes near the last target point. With the GDAE method, the robot tends to wander near the walls. Our method demonstrates superior navigation performance, indicating exceptional generalization ability that substantially minimizes the risk of collision. It enables the robot to handle high-dimensional inputs such

as laser scans relying on spatial-temporal state encoding using BiGRU. The combination of BiGRU and attention techniques, tailored to sensor limitations, optimizes decision-making in partially observable scenes.

6. Real-world experiment validation

6.1. Experimental setup

To evaluate the applicability of our memory-driven policy in practical settings, experiments were carried out in real-world scenarios. The robot used for the experiment was a Turtlebot3 fitted with a Hokuyo LiDAR sensor, which provides a 360° FoV and an odometer, which, in conjunction with LiDAR, were employed for simultaneous localization and mapping. In a scenario with non-human obstacles, the environment map was created using the Gmapping ROS package [33], and to determine the precise robot's position within the map, the AMCL ROS package [34] was used. To detect the human states, the Obstacle detector package [35] was adopted. Leveraging the sensor data such as map, localization, and human states, the proposed method running on a host computer equipped with Ubuntu 18.04 and ROS generated control commands for the robot, guiding it toward its established goal position. Fig. 6 depicts the proposed navigation framework which combines the sensory data, localization, and a local goal from the path planning module. These inputs are processed by the memory-driven policy to generate optimal velocity commands.

6.2. Results

Initially, as depicted in Fig. 7, the Turtlebot3 was deployed in a scenario with five humans, static objects, and a FoV of 360°. In the initial encounter, the robot made a slight leftward movement to avoid the human, and similarly, when the second human approached, the robot turned right, successfully navigating around the humans and the static obstacles in its path. The robot was then capable of avoiding the other humans coming from the front and of reaching its goal. In the experiments, 10 trial runs achieved a success rate of 95%, as is presented in Table 4.

As shown in Fig. 8, the robot's travel area increased and a greater number of static objects and humans were included. The robot stopped and immediately looked in a different direction to continue on the intended path when it detected a human approaching from the front. The robot anticipated collisions and navigated rapidly and safely to its destination. At times, the robot executed a complete turn before integrating new pathways toward its goal, consequently prolonging the time required to reach the objective. The robot performed remarkably well in avoiding obstacles and succeeded in all trials. The proposed policy successfully engaged with humans, even amid unpredictable human behavior and varying trajectories, while addressing the potential disruptions arising from the robot's dynamics and perception module.

6.3. Discussion on real-world experiments

The Turtlebot3 used in our experiments has certain hardware limitations. Powered by a Raspberry Pi 3, it experienced computational constraints that hindered its ability to process real-time interactions and execute actions quickly. To overcome this, we offloaded the processing of real-time data from Hokuyo LiDAR such as human states to a remote PC, which also derived from our method the robot's motion. With a maximum speed of 0.22 m/s, the Turtlebot3 might struggle to take prompt action or avoid obstacles if a human moves faster than the robot. The real-world tests were conducted as zero-shot experiments, meaning our policy was deployed without retraining or fine-tuning, enabling us to assess the generalization of our model. Additional constraints were revealed such as hardware fluctuations (e.g., battery and motor calibration) and the variability of human behavior. Despite these challenges, Turtlebot3 navigated its environment successfully and avoided collisions, demonstrating the potential of our approach for real-world applications.

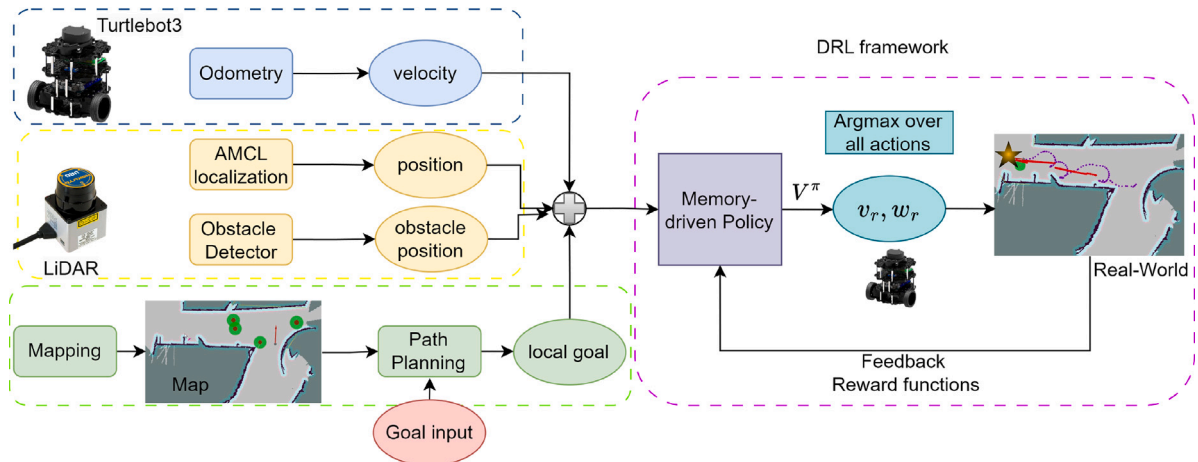


Fig. 6. The architecture of the experimental memory-driven framework. The system integrates sensory data from Turtlebot3 (velocity via odometry) and LiDAR (AMCL localization and obstacle detection). A pre-built map and goal input inform the path planning module, which outputs a local goal. The memory-driven policy leverages these inputs to produce optimal control actions (v_r, w_r) via a learned value function (V^π). The policy selects actions by evaluating all possible actions to ensure efficient and adaptive navigation. Reward functions continuously refine the policy for improved performance.



Fig. 7. Real-world Experiments with Hokuyo LiDAR Featuring 360° resolution: On the right, the RVIZ simulator from ROS is depicted, while on the left, a TurtleBot is positioned within a corridor, with human subjects situated in its path.

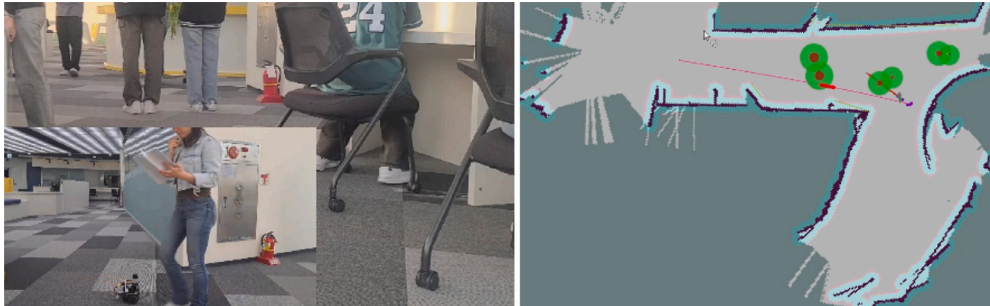


Fig. 8. Real-world experiments with Hokuyo LiDAR featuring 360° resolution: On the right, the RVIZ simulator from ROS is depicted, while on the left, a TurtleBot is positioned in an expanded area, encountering more than seven humans in its trajectory.

Table 4
Real-Time results in a crowded environment.

Number of humans	Goal distance = 6 m		Goal distance = 8 m	
	4	5	6	7
Success Rate (%)	97	95	95	94
Traveled Time (s)	58.75	48	71	79
Traveled Distance (m)	13.51	11.04	16.33	18.17

7. Limitations

Factors such as intersections, walls, corridors, and objects such as chairs and tables are not considered in the simulation. The robot

was trained in an environment that performs movements without constraints, which differs from real-world scenarios where physical objects and architectural features restrict movement. In the unstructured simulated scenarios conducted with fifteen humans, the robot with our proposed policy navigated and avoided collisions successfully. However, in narrow spaces, the robot's ability to navigate and avoid obstacles may be limited due to congestion. In addition, this could lead to situations where humans become occluded by the robot's sensors, generating additional challenges for the robot concerning path planning and decision-making.

To address these limitations, our future work will incorporate more structured environments, including walls, corridors, and intersections, to improve simulated real-world conditions. Additionally, we plan to enhance the robot's ability to anticipate human movements and infer

occluded agents in narrower passages.

8. Conclusion

This study proposes a learning-based method to enable a robot to navigate safely in pedestrian-rich partially observable scenarios. The developed BiGRU-based neural network captured and encoded the past behavior patterns of both humans and a robot. This process extracted environmental features, which, when coupled with attention mechanisms, mapped these features directly, and the proposed model was able to make better-informed decisions. Thorough simulation experiments were conducted to assess the policy's navigational performance. The simulation results indicate that the memory-driven overcomes existing navigation techniques in terms of success, collision, and discomfort rate, offering a safer strategy for guiding robots in environments with simulated realistic conditions such as narrow FoV of 120° and 360° with a short sensor range. The suggested policy was further implemented in a real robot, and field tests were conducted to evaluate the simulation-to-real performance. Our memory-driven policy demonstrates robustness across varying crowd densities and potential for generalizing to new environments, as validated by sim-to-real experiments with different pedestrian densities. To address the challenges that remain in unstructured or constrained environments with unpredictable human behavior. Future work aims to incorporate additional environmental constraints, such as walls and narrow corridors with high-density crowds, and a module to infer occlusions.

CRedit authorship contribution statement

Estrella Montero: Writing – original draft, Software, Methodology, Formal analysis, Conceptualization. **Nabih Pico:** Investigation, Formal analysis, Conceptualization. **Mitra Ghergherehchi:** Writing – review & editing, Visualization, Supervision, Investigation, Data curation, Conceptualization. **Ho Seung Song:** Writing – review & editing, Resources, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2022-00166733) and also supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2023-00221186).

Appendix A. Summary of symbols and definitions

See Table A.1.

Appendix B. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.jestch.2024.101942>.

Table A.1
Summary of symbols and Definitions.

Parameter	Description
$\mathbf{p}_r$	Position of the robot: $[x_r, y_r]$
$\mathbf{g}$	Goal Position: $[g_x, g_y]$
$\mathbf{v}_r$	Velocity of robot: $[v_{rx}, v_{ry}]$
$\mathbf{v}_m$	Velocity of human: $[v_{mx}, v_{my}]$
d_{rg}	Robot's distance to goal: $\ \mathbf{p}_r - \mathbf{g}\ _2$
d_m	Robot's distance to human: $\ \mathbf{p}_r - \mathbf{p}_m\ _2$
$\ \cdot\ _2$	Standard L_2 -norm (Euclidean distance)
V^π	Value function
π^*	Optimal policy
γ	Discount factor
$\mathbf{s}$	Robot state
$\mathbf{h}_s$	Human state
$\mathbf{j}_s$	Join state of system
$R(\mathbf{s}, \mathbf{a})$	Reward Function
d_{Ht}	Human's distance to FoV
$\mathbf{h}_{out}^r, \mathbf{h}_{out}^m$	Robot and human BiGRU's outputs
σ, τ	Intermediate feature representation
ϕ	Attention score

References

- [1] A. Loganathan, N.S. Ahmad, A systematic review on recent advances in autonomous mobile robot navigation, *Eng. Sci. Technol., Int. J.* 40 (2023) 101343.
- [2] S. Beycimen, D. Ignatyev, A. Zolotas, A comprehensive survey of unmanned ground vehicle terrain traversability for unstructured environments and sensor technology insights, *Eng. Sci. Technol., Int. J.* 47 (2023) 101457.
- [3] K. Wu, H. Wang, M.A. Esfahani, S. Yuan, Learn to navigate autonomously through deep reinforcement learning, *IEEE Trans. Ind. Electron.* 69 (5) (2021) 5342–5352.
- [4] E.E. Montero, H. Mutahira, N. Pico, M.S. Muhammad, Dynamic warning zone and a short-distance goal for autonomous robot navigation using deep reinforcement learning, *Complex Intell. Syst.* (2023) 1–18.
- [5] C. Chen, Y. Liu, S. Kreiss, A. Alahi, Crowd-robot interaction: Crowd-aware robot navigation with attention-based deep reinforcement learning, in: 2019 International Conference on Robotics and Automation, ICRA, IEEE, 2019, pp. 6015–6022.
- [6] T. Fan, X. Cheng, J. Pan, P. Long, W. Liu, R. Yang, D. Manocha, Getting robots unfrozen and unlost in dense pedestrian crowds, *IEEE Robot. Autom. Lett.* 4 (2) (2019) 1178–1185.
- [7] Y.F. Chen, M. Liu, M. Everett, J.P. How, Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning, in: 2017 IEEE International Conference on Robotics and Automation, ICRA, IEEE, 2017, pp. 285–292.
- [8] Y.J. Mun, M. Itkina, S. Liu, K. Driggs-Campbell, Occlusion-aware crowd navigation using people as sensors, in: 2023 IEEE International Conference on Robotics and Automation, ICRA, IEEE, 2023, pp. 12031–12037.
- [9] W. Shi, Y. Zhou, X. Zeng, S. Li, M. Benezit, Enhanced spatial attention graph for motion planning in crowded, partially observable environments, in: 2022 International Conference on Robotics and Automation, ICRA, IEEE, 2022, pp. 4750–4756.
- [10] M. Itkina, Y.J. Mun, K. Driggs-Campbell, M.J. Kochenderfer, Multi-agent variational occlusion inference using people as sensors, in: 2022 International Conference on Robotics and Automation, ICRA, IEEE, 2022, pp. 4585–4591.
- [11] C. van der Ploeg, T. Nyberg, J.M.G. Sánchez, E. Silvas, N. van de Wouw, Overcoming fear of the unknown: Occlusion-aware model-predictive planning for automated vehicles using risk fields, *IEEE Trans. Intell. Transp. Syst.* (2024).
- [12] L. Liu, Q. Xu, M. Zhang, H. Ji, Q. Zhao, A BiGRU based adaptive gain estimation for radar multi-target tracking, in: Chinese Conference on Pattern Recognition and Computer Vision, PRCV, Springer, 2023, pp. 405–417.
- [13] M. Huang, M. Zhu, Y. Xiao, Y. Liu, Bayonet-corpus: a trajectory prediction method based on bayonet context and bidirectional GRU, *Digit. Commun. Netw.* 7 (1) (2021) 72–81.
- [14] Y. Liu, T. Wang, D. Qiu, Multi-robot motion planning for dynamic environment based on deep reinforcement learning, in: 2024 IEEE 7th Advanced Information Technology, Electronic and Automation Control Conference, IAEAC, vol. 7, IEEE, 2024, pp. 892–896.
- [15] J. Van Den Berg, S.J. Guy, M. Lin, D. Manocha, Reciprocal n-body collision avoidance, in: Robotics Research: The 14th International Symposium ISRR, Springer, 2011, pp. 3–19.
- [16] G. Ferrer, A. Garrell, A. Sanfeliu, Robot companion: A social-force based approach with human awareness-navigation in crowded environments, in: 2013 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE, 2013, pp. 1688–1694.

- [17] P. Chintam, T. Lei, B. Osmanoglu, Y. Wang, C. Luo, Informed sampling space driven robot informative path planning, *Robot. Auton. Syst.* (2024) 104656.
- [18] T. Fan, P. Long, W. Liu, J. Pan, Distributed multi-robot collision avoidance via deep reinforcement learning for navigation in complex scenarios, *Int. J. Robot. Res.* 39 (7) (2020) 856–892.
- [19] A.J. Sathyamoorthy, U. Patel, T. Guan, D. Manocha, Frozone: Freezing-free, pedestrian-friendly navigation in human crowds, *IEEE Robot. Autom. Lett.* 5 (3) (2020) 4352–4359.
- [20] N. Pico, B. Lee, E. Montero, M. Tadese, E. Auh, M. Doh, H. Moon, Static and dynamic collision avoidance for autonomous robot navigation in diverse scenarios based on deep reinforcement learning, in: 2023 20th International Conference on Ubiquitous Robots, UR, IEEE, 2023, pp. 281–286.
- [21] S. Yao, G. Chen, Q. Qiu, J. Ma, X. Chen, J. Ji, Crowd-aware robot navigation for pedestrians with multiple collision avoidance strategies via map-based deep reinforcement learning, in: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, IEEE, 2021, pp. 8144–8150.
- [22] X. Huang, H. Deng, W. Zhang, R. Song, Y. Li, Towards multi-modal perception-based navigation: A deep reinforcement learning method, *IEEE Robot. Autom. Lett.* 6 (3) (2021) 4986–4993.
- [23] M. Everett, Y.F. Chen, J.P. How, Motion planning among dynamic, decision-making agents with deep reinforcement learning, in: 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, IEEE, 2018, pp. 3052–3059.
- [24] N. Pico, E. Montero, M. Vanegas, J.M. Erazo Ayon, E. Auh, J. Shin, M. Doh, S. Park, H. Moon, Integrating radar-based obstacle detection with deep reinforcement learning for robust autonomous navigation, *Appl. Sci.* 15 (1) (2024) 295.
- [25] S.L. Dai, K. Lu, J. Fu, Adaptive finite-time tracking control of nonholonomic multirobot formation systems with limited field-of-view sensors, *IEEE Trans. Cybern.* 52 (10) (2021) 10695–10708.
- [26] M. Santilli, P. Mukherjee, R.K. Williams, A. Gasparri, Multirobot field of view control with adaptive decentralization, *IEEE Trans. Robot.* 38 (4) (2022) 2131–2150.
- [27] M. Bouton, A. Nakhaei, K. Fujimura, M.J. Kochenderfer, Safe reinforcement learning with scene decomposition for navigating complex urban environments, in: 2019 IEEE Intelligent Vehicles Symposium, IV, IEEE, 2019, pp. 1469–1476.
- [28] V.R. Miranda, A.A. Neto, G.M. Freitas, L.A. Mozelli, Generalization in deep reinforcement learning for robotic navigation by reward shaping, *IEEE Trans. Ind. Electron.* (2023).
- [29] N. Pico, J. Lee, E. Montero, E. Auh, M. Tadese, J. Jeon, M.S. Alvarez-Alvarado, H. Moon, Enhancing autonomous robot navigation based on deep reinforcement learning: Comparative analysis of reward functions in diverse environments, in: 2023 23rd International Conference on Control, Automation and Systems, ICCAS, IEEE, 2023, pp. 1415–1420.
- [30] A. Alipanah, S.A.A. Moosavian, An improvement on mapless navigation with deep reinforcement learning: A reward shaping approach, in: 2022 10th RSI International Conference on Robotics and Mechatronics, ICRoM, IEEE, 2022, pp. 261–266.
- [31] H. Li, J. Qin, Q. Liu, C. Yan, An efficient deep reinforcement learning algorithm for mapless navigation with gap-guided switching strategy, *J. Intell. Robot. Syst.* 108 (3) (2023) 43.
- [32] W. Zhang, N. Liu, Y. Zhang, Learn to navigate maplessly with varied LiDAR configurations: A support point-based approach, *IEEE Robot. Autom. Lett.* 6 (2) (2021) 1918–1925.
- [33] B. Gerkey, Gmapping ros package, 2024, <https://wiki.ros.org/gmapping/>. (Accessed 1 April 2024).
- [34] B. Gerkey, Amcl ros package, 2022, <https://wiki.ros.org/amcl/>. (Accessed 1 April 2024).
- [35] M. Przybyla, Obstacle detector ros package, 2022, <https://github.com/tysik/obstacledetector>. (Accessed 1 April 2024).
- [36] R. Cimurs, I.H. Suh, J.H. Lee, Goal-driven autonomous exploration through deep reinforcement learning, *IEEE Robot. Autom. Lett.* 7 (2) (2021) 730–737.